

海康嵌入式笔试题

海康嵌入式笔试题

1. 请问 TCP/IP 协议分为哪几层？FTP 协议属于哪一层？

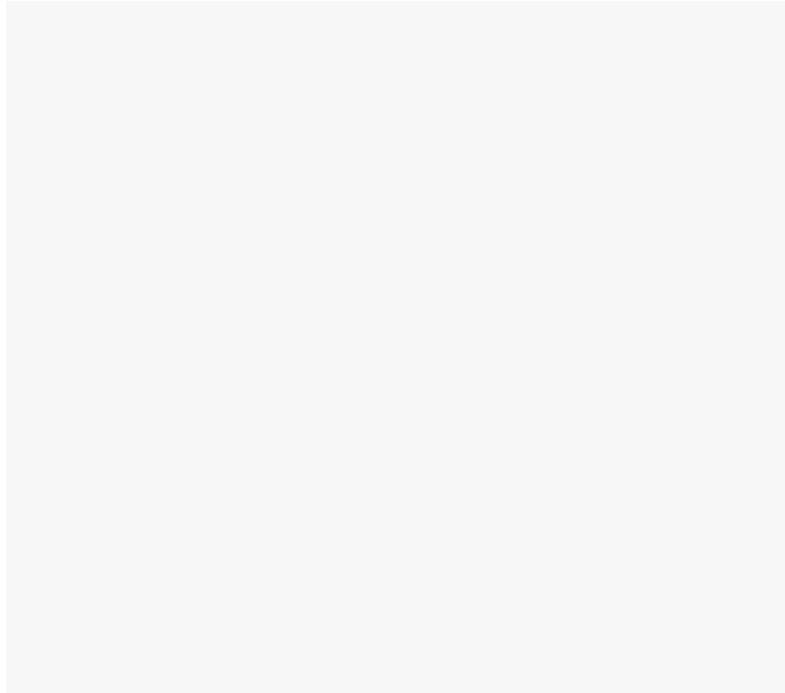
答：可以分为 物理层，数据链路层，网络层，传输层，应用层

应用层：应用程序间沟通的层，如简单电子传输（SMTP）、文件传输协议（FTP）、网络远程访问协议（**Telnet**）等。

传输层：在此层中，它提供了节点间的数据传送，应用程序之间的通信服务，主要功能是数据格式化、数据确认和丢失重传等。如传输控制协议（TCP）、用户数据报协议（UDP）等，TCP 和 UDP 给数据包加入传输数据并把它传输到下一层中，这一层负责传送数据，并且确定数据已被送达并接收。

互连网络层：负责提供基本的数据封包传送功能，让每一块数据包都能够到达目的的主机（但不检查是否被正确接收），如网际协议（IP）。如 ARP 是地址解析协议，在这一层的时候吧 IP 地址转换为物理地址

网络接口层（主机-网络层）：接收 IP 数据报并进行传输，从网络上接收物理帧，抽取 IP 数据报转交给下一层，对实际的网络媒体的管理，定义如何使用实际网络（如 Ethernet、Serial Line 等）来传送数据。



另外补充一下 OSI 的七层所对应的协议：

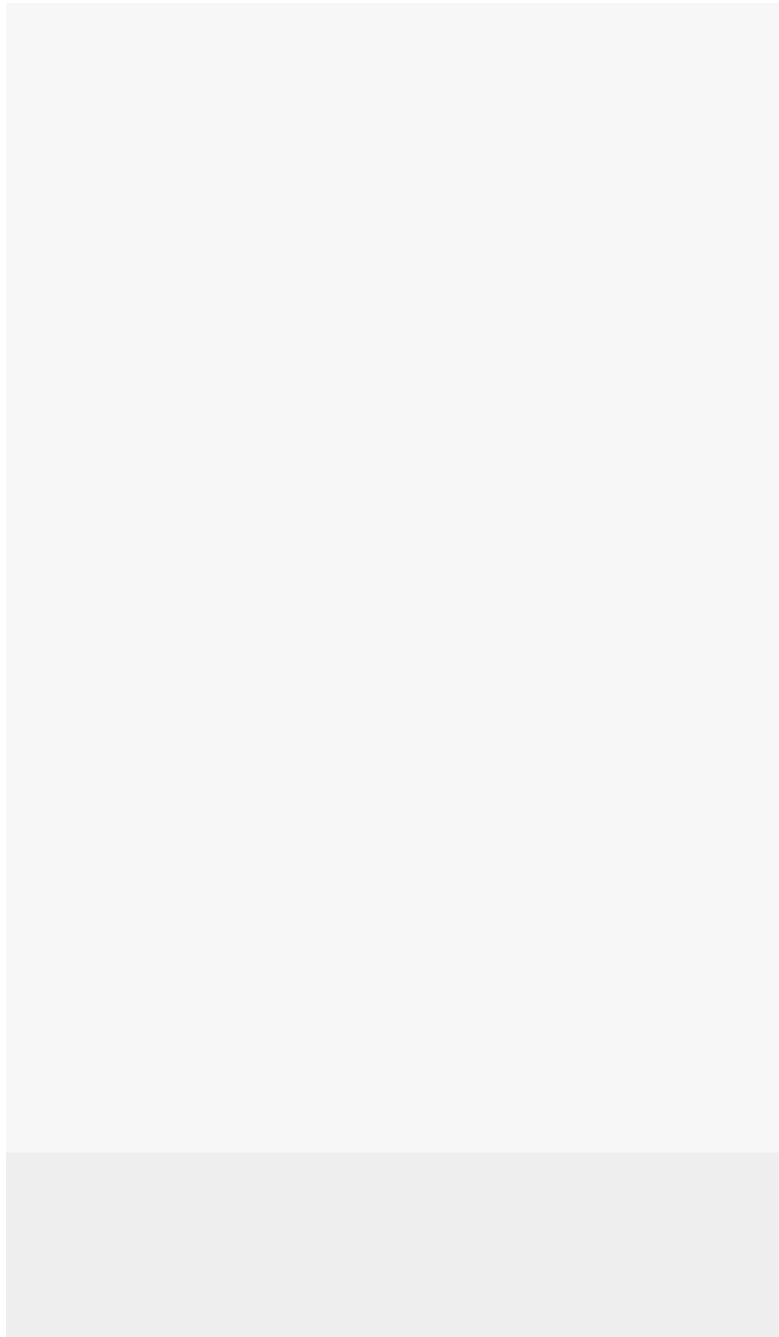
应用层 (Application)：

应用程序网关 (application gateway) Telnet：远程登录
(在应用层连接两部分应用程序)
FTP (File Transfer Protocol)：文件传输协议
HTTP (Hyper Text Transfer Protocol)：超文本传输协议
SMTP (Simple Mail Transter Protocol)：简单传输协议
POP3 (Post Office Ptotocol)：邮局协议
SNMP (Simple Network Mangement Protocol) 简单网络管理协议
DNS (Domain Name System)：域名系统

传输层 (Transport)：

传输网关 (transport gateway)
TCP (Transmission Control Potocol)：传输控制协议
(在传输层连接两个网络)
UDP (User Data Potocol)：用户数据协议

专业 WORD.



网络层（Internet）：

多协议路由器（multiprotocol router）

IP（Internet Protocol）：网络协议 （在异构网络间转发分组）

ARP（Address Resolution Protocol）：地址解析协议

RARP（Reverse Address Resolution Protocol）：逆地址解析协议

ICMP（Internet Control Message Protocol）：因特网控制消息协议

IGMP（Internet Group Manage Protocol）：因特网组管理协议

BOOTP（Bootstrap）：可选安全启动协议

数据链路层 (Data Link) :

网桥 (bridge) 交换机 (switcher)

HDLC (High Data Link Control) : 高级数据链路控制

(在 LAN 之间存储-转发数据链路帧)

SLIP (Serial Line IP) : 串行线路 IP

PPP (Point-to-Point Protocol) : 点到点协议 802.2 等

物理层 (Physical) :

中继器 (repeater) 集线器 (hub)

(放大或再生弱的信号, 在两个电缆段之间复制每一个比特)

再回过头来看, ftp 是用 tcp 写的一个文件传输协议, 明显就是在应用层了

=====

2. 在网络应用中, 函数 htons, htonl, ntohs, ntohl 的作用?

htons: 把短整型的主机字节顺序转变为网络字节顺序

其它依次类推

htons 一般用来转变端口, htonl 一般用来转 IP

网络字节顺序是先高位字节, 再低位字节; 主机字节顺序则视 cpu 而定

干脆扩展一点

```
#include<unistd.h>
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
#include<netinet/in.h>
```

```
#include<sys/socket.h>
```

专业 WORD.

```
#include<sys/types.h>
```

```
int main()
```

```
{
```

```
    char str[]="255.255.255.255";
```

```
    in_addr_t r1,r2,r3;
```

```
    struct in_addr inp;
```

```
    r1=inet_addr(str);
```

```
    if(r1==-1)
```

```
    {
```

```
        printf("inet_addr return -1 when 255.255.255.255\n");
```

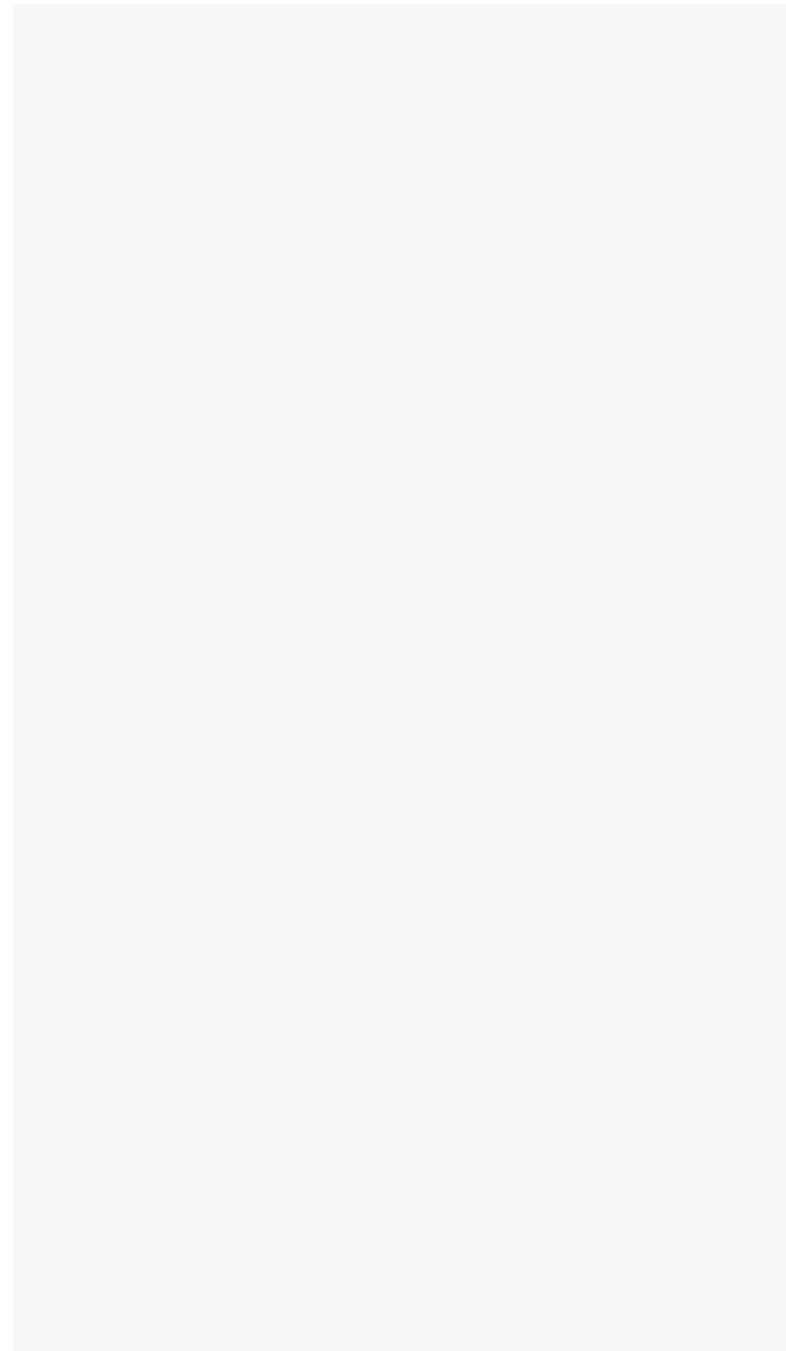
```

    }
    else
    {
        printf("inet_addr: ip=%lu\n", ntohl(r1));
    }
    r2=inet_network(str);
    if(r2==-1)
    {
        printf("inet_network return -1 when 255.255.255.2
55\n");
    }
    else
    {
        printf("inet_network: ip=%lu\n", r2);
    }
    r3=inet_aton(str, &inp);
    if(r3==0)
    {
        printf("inet_aton return -1 when 255.255.255.255\n
");
    }
    else
    {
        printf("inet_aton: ip=%lu\n", ntohl(inp.s_addr));
    }
    return 0;
}

```

3. 在 c 语言中，static 函数与普通函数有什么区

专业 WORD.



别？

1. 全局静态变量

在全局变量之前加上关键字 `static`，全局变量就被定义成为一个全局静态变量。

1) 存中的位置：静态存储区（静态存储区在整个程序运行期间都存在）。

2) 初始化：未经初始化的全局静态变量会被程序自动初始化为 0（自动对象的值是任意的，除非他被显示初始化）。

3) 作用域：全局静态变量在声明他的文件之外是不可见的。准确地讲从定义之处开始到文件结尾。

定义全局静态变量的好处：

<1>不会被其他文件所访问，修改。

<2>其他文件中可以使用相同名字的变量，不会发生冲突。

2. 局部静态变量

在局部变量之前加上关键字 `static`，局部变量就被定义成为一个局部静态变量。

1) 存中的位置：静态存储区。

2) 初始化：未经初始化的全局静态变量会被程序自动初始化为 0（自动对象的值是任意的，除非他被显示初始化）。

3) 作用域：作用域仍为局部作用域，当定义它的函数或者语句块结束的时候，作用域随之结束。

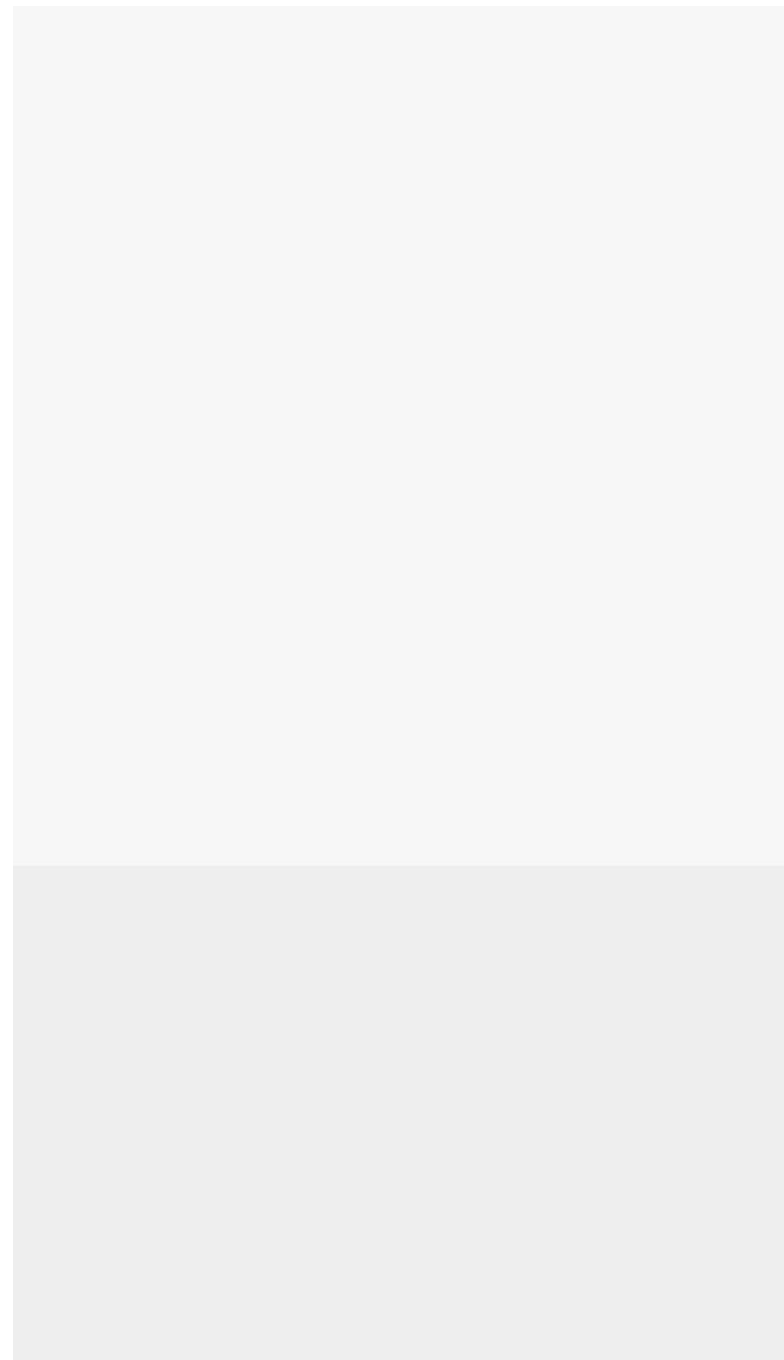
注：当 `static` 用来修饰局部变量的时候，它就改变了局部变量的存储位置，从原来的栈中存放改为静态存储区。考试,大提示局部静态变量在离开作用域之后，并没有被销毁，而是仍然驻留在存当中，直到程序结束，只不过我们不能再对他进行访问。

当 `static` 用来修饰全局变量的时候，它就改变了全局变量的作用域（在声明他的文件之外是不可见的），但是没有改变它的存放位置，还是在静态存储区中。

3. 静态函数

在函数的返回类型前加上关键字 `static`，函数就被定义成为静态函数。

专业 WORD.



函数的定义和声明默认情况下是 `extern` 的，但静态函数只是在声明他的文件当中可见，不能被其他文件所用。

定义静态函数的好处：

<1> 其他文件中可以定义相同名字的函数，不会发生冲突。

<2> 静态函数不能被其他文件所用。

存储说明符 `auto`, `register`, `extern`, `static`，对应两种存储期：自动存储期和静态存储期。

`auto` 和 `register` 对应自动存储期。具有自动存储期的变量在进入声明该变量的程序块时被建立，它在该程序块活动时存在，退出该程序块时撤销。

关键字 `extern` 和 `static` 用来说明具有静态存储期的变量和函数。用 `static` 声明的局部变量具有静态存储持续期（static storage duration），或静态围（static extent）。虽然他的值在函数调用之间保持有效，但是其名字的可视性仍限制在其局部域。静态局部对象在程序执行到该对象的声明处时被首次初始化。

扩展分析：

术语 `static` 有着不寻常的历史. 起初，在 C 中引入关键字 `static` 是为了表示退出一个块后仍然存在的局部变量。随后，`static` C 中有了第二种含义：用来表示不能被其它文件访问的全局变量和函数。为了避免引入新的关键字，所以仍使用 `static` 关键字来表示这第二种含义。最后，C++重用了这个关键字，并赋予它与前面不同的第三种含义：表示属于一个类而不是属于此类的任何特定对象的变量和函数(与 Java 中此关键字的含义相同)。

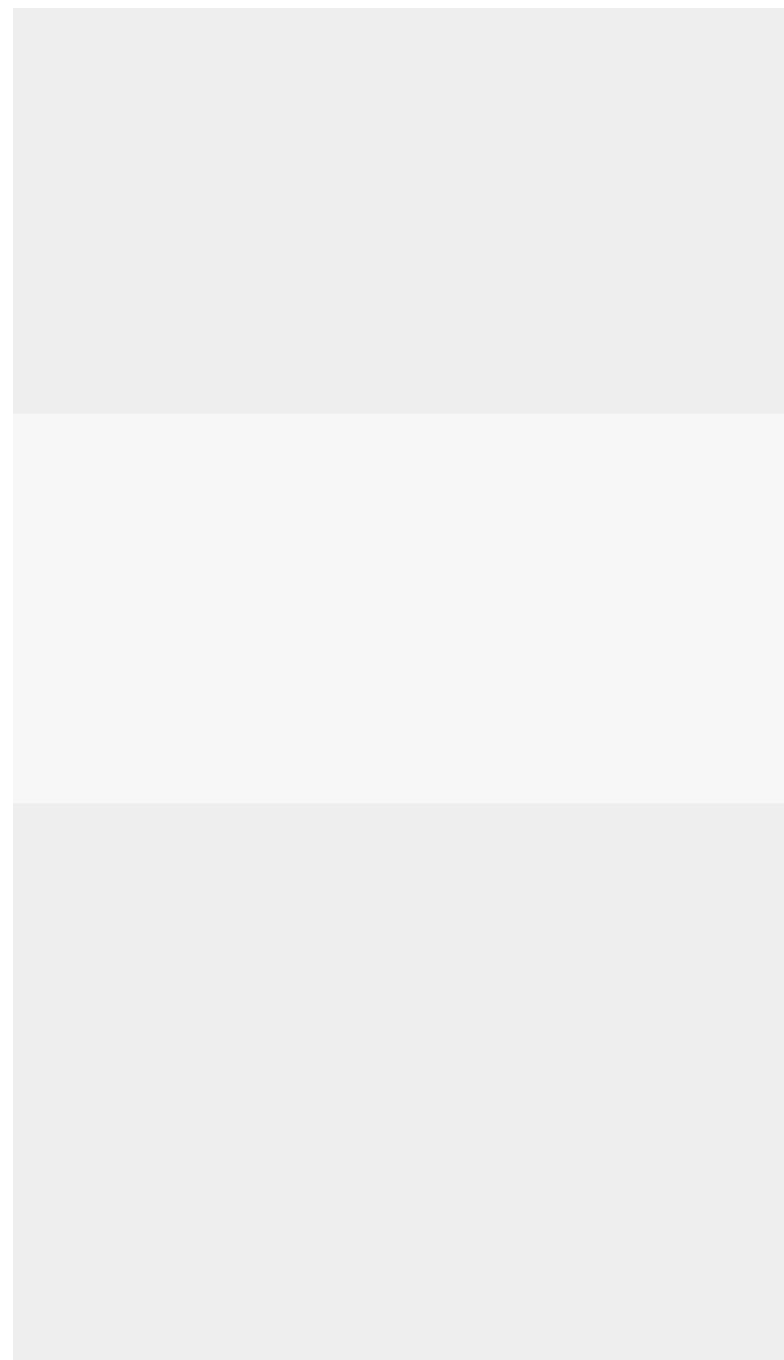
写个例子，比对一下

```
#include <stdio.h>
#include <stdlib.h>

extern int a;//声明
static int b=5;

a=0;//初始化
void func1()
{
    printf("a=%d;b=%d\n",a,b);
    return ;
}

void func2()
{
    static int i=333;
    专业 WORD.
```



```
        i++;  
        printf("i=%d\n",i);  
        return ;  
    }
```

```
int main()  
{  
    a=20;  
    func1();  
    func2();  
    func2();  
}
```

```
    return 0;
}
```

输出为:

a=20;b=5

i=334

i=335

总的来说，静态的东西就是只在本文件中可见，而且在本文件中保持，它只初始化一次，存在静态存储区中，再对它进行初始化不会引起错误，但是没有用了

=====

4. 请实现存复制函数

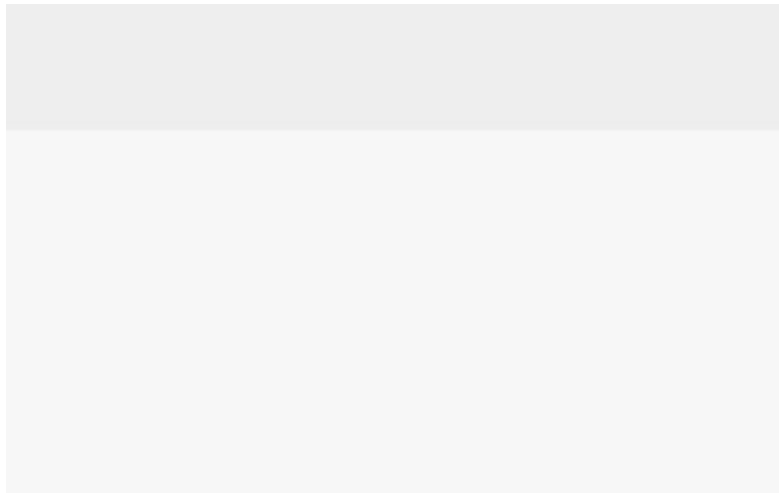
void memcpy (void *dst,void *src,int size) ?

网上搜了一下

```
void *MyMemCopy(void *dest,const void *src,size_t count)
{
    char *pDest=static_cast<char *>(dest);
    const char *pSrc=static_cast<const char *>(src);

    if( pDest>pSrc && pDest<pSrc+count )
    {
        for(size_t i=count-1; i<=0; ++i)
        {
            pDest[i]=pSrc[i];
        }
    }
    else
    {
        for(size_t i=0; i<count; ++i)
        {
            pDest[i]=pSrc[i];
        }
    }
}
```

专业 WORD.



```
}
```

```
return pDest;
```

```
}
```

哎，搞 IT 真的没有前途，你看，人都这么聪明的？！

就这还考虑到了，幸好，我还不是笨到家，还能看得懂

稍微解释下：

如果是这个情况：

src -----

dst -----

说明两个在存区有重叠的地方。

如果用 `dst[0]=src[0]`

那么说明 `src` 的某块会被弄脏掉，怎么办？从后面拷起就不会有问题，`dst[size-1]=src[size-1]`

如果没有重叠的话就从头拷起

如程序所见